

Jogo de Cartas em Rede

Adaptação do jogo *Exploding Kittens* cliente-servidor TCP em C++

Jogo multijogador de terminal: um servidor hospeda o lobby e a partida; 2 a 5 clientes se conectam, escolhem um nome, ficam *prontos* e jogam por turnos. Vence o último jogador que não explodir.

1 Grupo

Alexandre Coura Serravite	15507531
Bruno Baremaker Moraes	15443854
Felipe Freitas Maia	15451360
Matheus Watanabe de Vilhena	15479767
Rafael Olandoski de Oliveira	15485382

2 Ambiente

- **Sistema operacional:** Linux Mint 22.2 (base Ubuntu 24.04 LTS)
- **Compilador:** g++ (GCC) 13.3.0, padrão C++17
- **Dependências:** biblioteca padrão e APIs POSIX (`socket`, `poll`, `termios`, `sigaction`); **não** compila no Windows.

3 Estrutura

controle/	Servidor, ServidorTCP, Partida, Cliente, ClienteTCP
jogo/	Jogador, Carta
interacao/	Interface (entrada/saída do terminal)
biblio/	cabeçalhos
bin/	executáveis gerados

4 Compilação e execução

```
make -f Makefile           # gera bin/servidor e bin/cliente

./bin/servidor [porta]     # padrao: 5000; pede o nome do servidor
./bin/cliente [host] [porta] # padrao: localhost 5000
```

Use um terminal para o servidor e um por jogador. O servidor encerra com **Ctrl+C**.

5 Como jogar

Fluxo: tela inicial (ESC sai, qualquer outra tecla entra) → nome (até 12 letras/números, único, sem distinguir maiúsculas) → lobby (P+Enter alterna pronto, S+Enter sai) → partida. Ela começa quando há de 2 a 5 jogadores e todos estão prontos.

Regras: cada jogador começa com 1 DEFUSE + 7 cartas. No seu turno, joga cartas (opcional) e termina **comprando**. Ao comprar uma bomba, o DEFUSE a devolve ao baralho em posição aleatória; sem ele, o jogador é eliminado. Toda carta de ação ou combo abre uma janela em que os outros podem cancelá-la com NAO.

ATACAR	próximo joga 2 turnos	FUTURO	vê as 3 próximas cartas
PULAR	termina o turno sem comprar	NAO	cancela uma ação (reação)
FAVOR	alvo entrega uma carta à sua escolha	GATO1–GATO5	só em combos
EMBARALHAR	embaralha o baralho		

Combos (cartas do mesmo tipo): com 2 cartas rouba-se uma carta *aleatória* de um jogador; com 3, escolhe-se o jogador *e o tipo* da carta (falha se o alvo não a tiver).

Comando (não diferencia caixa)	Uso
COMPRAR	compra e encerra o turno
JOGAR <n>	joga a carta de índice n da mão
COMBO <n> <n> [<n>]	combo de 2 ou 3 cartas (índices distintos)
JOGAR_NAO / PASSO	responde à janela de reação
nome / número / tipo	escolhe alvo, carta a entregar ou tipo a roubar (quando solicitado)
MESA, DESCARTE, SAIR	redesenha a mesa, mostra o descarte, sai

6 Protocolo

TCP/IPv4, mensagens de texto **terminadas em** `\textbackslash n`.

7 Verificações de falha

Conexão

- **Recusa:** lobby cheio (5) ou partida em andamento $\Rightarrow$ `ERRO LOBBY_CHEIO` e `close()`.
- **Desconexão de cliente** (`recv() <= 0`): o socket é fechado, o jogador sai da lista e é eliminado da partida (que pode terminar), e o lobby é atualizado

Transmissão

- **Envio completo:** `send()` em loop até enviar todos os bytes, repetindo em `EINTR`; retorno `<= 0` é falha. `MSG_NOSIGNAL` evita `SIGPIPE` em socket fechado.
- **Delimitação:** como o TCP é um fluxo de bytes, os dados são acumulados em buffer e só **linhas completas** são processadas (trata mensagens fragmentadas ou coladas; remove `\textbackslash r`).
- **Abuso:** mais de 4096 bytes sem `\textbackslash n` $\Rightarrow$ conexão encerrada.
- **Concorrência:** mutex no servidor; no cliente, `mtxEnvio` serializa `send()` e `mtx` protege o estado. Os *callbacks* rodam fora do lock (evita deadlock).
- **Validação:** nomes, comandos desconhecidos, índices, combos, alvos e vez do jogador são checados no servidor; o cliente ignora mensagens desconhecidas e descarta itens malformados do LOBBY